

Algoritmi

Riccardo Torre

24 marzo 2026

Indice

1	Strutture dati	1
1.1	Stack minimo/coda minima	1
1.1.1	Modifica dello stack	1
1.1.2	Modifica della coda – metodo 1	2
1.1.3	Modifica della coda – metodo 2	3
1.1.4	Modifica della coda – metodo 3	3
1.1.5	Trovare il minimo per tutti i sottoarray di lunghezza fissa	4
1.2	Sparse table	5
1.2.1	Intuizione	6
1.2.2	Precalcolo	6
1.2.3	Query sulla soma degli intervalli	7
1.2.4	Query minime di intervallo (RMQ)	7
1.2.5	Limitazioni	8
1.3	Unione di insiemi disgiunti	8
1.3.1	Costruire una struttura dati efficiente	8
1.3.2	Implementazione ingenua	9
1.3.3	Ottimizzazione della compressione del percorso	9

1 Strutture dati

1.1 Stack minimo/coda minima

1.1.1 Modifica dello stack

Per trovare l'elemento più piccolo in uno stack in $O(1)$ mantenendo il comportamento asintotico per aggiunta e rimozione di elementi, si memorizzano delle coppie.

Listing 1: aggiungere un elemento.

```
1 int new_min = st.empty() ? new_elem : min(new_elem, st.top().second);
2 st.push({new_elem, new_min});
```

Listing 2: rimuovere un elemento.

```
1 int removed_element = st.top().first;
2 st.pop();
```

Listing 3: trovare il minimo.

```
1 int minimum = st.top().second;
```

Lo stack ha un comportamento LIFO.

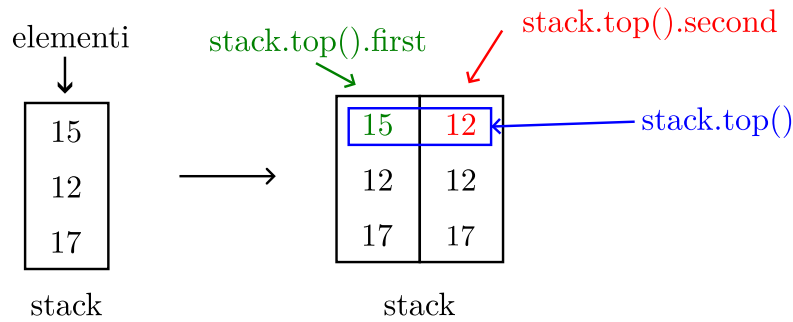


Figura 1: l'elemento minimo viene calcolato ad ogni inserimento e salvato sulla seconda colonna dell'elemento in cima allo stack (in rosso).

1.1.2 Modifica della coda – metodo 1

La coda ha un comportamento FIFO. Non memorizza tutti gli elementi: vengono memorizzati solo quelli necessari per determinare il minimo. La coda viene mantenuta in *ordine non decrescente*¹ (il valore più piccolo sarà memorizzato nella testa). Il minimo effettivo deve sempre essere mantenuto nella coda.

Listing 4: implementazione della coda.

```
1 deque<int> q;
```

Listing 5: trovare il minimo.

```
1 int minimum = q.front();
```

Prima di aggiungere un nuovo elemento nella coda è sufficiente effettuare un “taglio”: vengono rimossi tutti gli elementi finali della coda che sono più grandi del nuovo elemento e successivamente si aggiunge il nuovo elemento alla coda. In questo modo viene mantenuto l'ordine non decrescente e non viene perso l'elemento corrente se in qualsiasi passaggio successivo è l'elemento minimo. Tutti gli elementi che vengono rimossi non potranno mai essere primi.

Listing 6: aggiungere un elemento.

```
1 //dal fondo della coda si stanno rimuovendo tutti gli elementi di q maggiori
  di new_element
2 while (!q.empty() && q.back() > new_element)
3   q.pop_back();
4   q.push_back(new_element);
```

Quando si vuole estrarre un elemento dalla testa, potrebbe non essere presente se è stato rimosso in precedenza con un elemento più piccolo. Pertanto quando si elimina un elemento da una coda bisogna conoscerne il valore. Se la testa della coda ha lo stesso valore, può essere rimossa in sicurezza, altrimenti non viene apportata nessuna modifica.

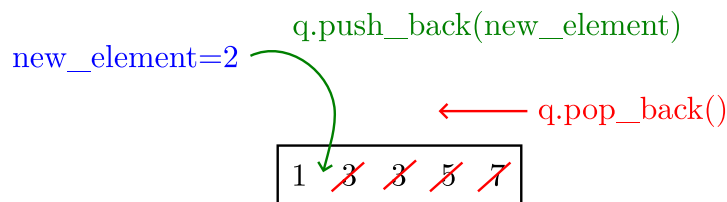
Listing 7: rimuovere un elemento.

```
1 if (!q.empty() && q.front() == remove_element)
2   q.pop_front();
```

Tutte queste operazioni richiedono $O(1)$.

¹cioè $\forall i \in [0, \text{coda.length()} - 1]: \text{coda}[i] \leq \text{coda}[i+1]$

La coda prima dell'inserimento



La coda dopo l'inserimento



Figura 2: effetto dell'inserimento di un nuovo elemento sulla coda.

1.1.3 Modifica della coda – metodo 2

Questa è una modifica del metodo 1. Si memorizza l'indice per ciascun elemento della coda, ricordando anche quanti elementi sono stati già aggiunti e rimossi.

Listing 8: implementazione della coda con contatori di elementi aggiunti e rimossi.

```
1 deque<pair<int, int>> q;
2 int cnt_added = 0;
3 int cnt_removed = 0;
```

Listing 9: trovare il minimo.

```
1 int minimum = q.front().first;
```

Listing 10: aggiungere un elemento.

```
1 while (!q.empty() && q.back().first > new_element)
2     q.pop_back();
3     q.push_back({new_element, cnt_added});
4     cnt_added++;
```

La modifica più rilevante è in questa porzione di codice (compararla con quella del metodo 1).

Listing 11: rimuovere un elemento.

```
1 if (!q.empty() && q.front().second == cnt_removed)
2     q.pop_front();
3     cnt_removed++;
```

1.1.4 Modifica della coda – metodo 3

Questo è un altro metodo per trovare il minimo in $O(1)$ modificando una coda. È più complicato da implementare ma permette di memorizzare tutti gli elementi e di rimuovere un elemento dalla parte anteriore senza conoscerne il valore. L'idea è di ridurre il problema al problema degli stack, simulando una coda. Si creano due pile, `s1` e `s2`. Si aggiungono nuovi elementi allo stack `s1`. e si rimuovono da `s2`. Se in qualsiasi momento `s2` è vuoto, si spostano tutti gli elementi da `s1` a `s2` (che inverte l'ordine degli elementi). Infine, per trovare il minimo in una coda è sufficiente trovare il minimo di entrambi gli stack.

Si eseguono tutte le operazioni in $O(1)$ in media (ogni elemento verrà aggiunto una volta allo stack `s1`, una volta trasferito a `s2` e una volta estratto da `s2`).

Listing 12: implementazione.

```
1 stack<pair<int, int>> s1, s2;
```

Listing 13: trovare il minimo.

```
1 if (s1.empty() || s2.empty())
2     minimum = s1.empty() ? s2.top().second : s1.top().second;
3 else
4     minimum = min(s1.top().second, s2.top().second);
```

Listing 14: aggiungi elemento.

```
1 int minimum = s1.empty() ? new_element : min(new_element, s1.top().second);
2 s1.push({new_element, minimum});
```

Listing 15: rimozione di un elemento.

```
1 if (s2.empty()) {
2     while (!s1.empty()) {
3         int element = s1.top().first;
4         s1.pop();
5         int minimum = s2.empty() ? element : min(element, s2.top().second);
6         s2.push({element, minimum});
7     }
8 }
9 int remove_element = s2.top().first;
10 s2.pop();
```

1.1.5 Trovare il minimo per tutti i sottoarray di lunghezza fissa

Si supponga di ricevere un array A di lunghezza N e un dato M tale che $M \leq N$. Si deve trovare il minimo di ogni sottoarray di lunghezza M in questo array, cioè si trovi:

$$\min_{0 \leq i \leq M-1} A[i], \min_{1 \leq i \leq M} A[i], \min_{2 \leq i \leq M+1} A[i], \dots, \min_{N-M \leq i \leq N-1} A[i]$$

Questo problema deve essere risolto in tempo lineare, cioè $O(n)$.

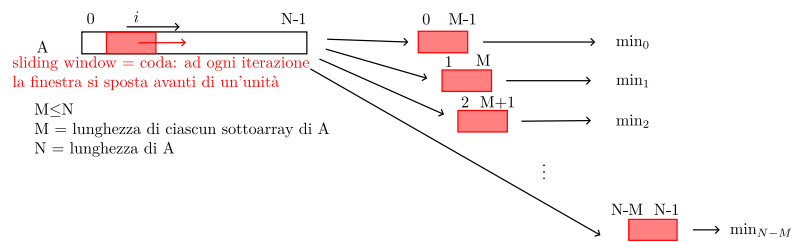


Figura 3: sottoarray di dimensione M .

È possibile utilizzare una qualsiasi delle tre code modificate per risolvere il problema.

1. Seleziona la finestra.
2. Aggiungi alla coda tutti gli elementi della finestra.
3. Ricava il minimo.
4. Rimuovi il minimo.

5. Sposta la finestra in avanti di uno.

Poiché tutte le operazioni sulla coda richiedono tempo costante in media, la complessità complessiva dell'algoritmo risulta $O(n)$.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  deque<int> q;
5  void print() {
6      for (auto i : q) {
7          cout << i << " ";
8      }
9      cout << "\n";
10 }
11 void addElement(int new_element) {
12     // per il massimo basta invertire il minore con il maggiore nella guardia
13     // del while
14     while (!q.empty() && q.back() > new_element) {
15         q.pop_back();
16     }
17     q.push_back(new_element);
18 }
19
20 int getMin() { return q.front(); }
21
22 void rmElement(int remove_element) {
23     if (!q.empty() && q.front() == remove_element) {
24         q.pop_front();
25     }
26 }
27
28 int main() {
29     vector<int> v = {3, 1, 4, 2, 5};
30     int M = 2;
31     int N = v.size();
32     vector<int> res;
33
34     for (int i = 0; i <= N - M; i++) {
35         for (int j = 0; j < M; j++) {
36             addElement(v.at(i + j));
37             // cout << "i: " << i << "\nj: " << j << "\ni+j: " << i + j << "\n";
38         }
39         res.push_back(getMin());
40
41         rmElement(getMin());
42     }
43     cout << "Risultato: ";
44     for (auto i : res) {
45         cout << i << " ";
46     }
47     cout << "\n";
48     return 0;
49 }
```

1.2 Sparse table

Una sparse table è una struttura dati che consente di rispondere a query di intervallo, rispondendo nella maggior parte dei casi in $O(\log n)$, a la sua vera potenza è rispondere a query con intervallo minimo/massimo in $O(1)$. L'unico inconveniente di questa struttura dati è che può

essere utilizzata solo su *matrici immutabili*. Ciò significa che la matrice non può essere modificata tra due query. Se un elemento della matrice cambia, è necessario ricalcolare l'intera struttura dati.

1.2.1 Intuizione

Qualsiasi numero non negativo può essere rappresentato in modo univoco come somma di potenze decrescenti di due. Questa è solo una variante della rappresentazione binaria di un numero. Ad esempio $13 = (1101)_2 = 8 + 4 + 1$. Per un certo numero x possono essere al massimo $\lceil \log_2 x \rceil$ addendi.

Con lo stesso ragionamento, qualsiasi intervallo può essere rappresentato in modo univoco come un'unione di intervalli con lunghezze che sono potenze decrescenti di due. Ad esempio $[2, 14] = [2, 9] \cup [10, 13] \cup [14, 14]$, dove l'intervallo completo ha lunghezza 13 e i singoli intervalli hanno rispettivamente lunghezza 8, 4 e 1. E anche qui l'unione è composta al massimo da $\lceil \log_2(\text{lunghezza dell'intervallo}) \rceil$ molti intervalli.

Listing 16: in python3 si calcolerebbe in questo modo.

```
1 import math
2 math.ceil(math.log(13,2)) # risulta 4 →  $\lceil \log_2 13 \rceil$ 
```

L'idea alla base delle tabelle sparse è quella di precalcolare tutte le risposte per query di intervallo con potenza di due lunghezze. Successivamente è possibile rispondere a una query di intervallo diverso suddividendo l'intervallo in intervalli con potenza di due lunghezze, cercando le risposte precalcolate e combinandole per ricevere una risposta completa.

1.2.2 Precalcolo

Si utilizzerà un array bidimensionale per memorizzare le risposte alle query precalcolate. `st[i][j]` memorizzerà la risposta per l'intervallo $[j, j + 2^i - 1]$ di lunghezza 2^i . La dimensione dell'array bidimensionale sarà $(K - 1) \times MAXN$, dove $MAXN$ è la lunghezza massima possibile dell'array. K deve soddisfare $K \geq \lceil \log_2 MAXN \rceil$, perché $2^{\lceil \log_2 MAXN \rceil}$ è la più grande potenza di due gamme, che si deve supportare. Per array di lunghezza ragionevole ($\leq 10^7$ elementi), $K = 25$ è un buon rapporto qualità-prezzo.

Si importa $MAXN$ come il numero di colonne della matrice per consentire accessi consecutivi alla memoria (cache).

```
1 int st[k+1][MAXN]; //ricorda che questa notazione significa nome_matrice[righe][colonne]
```

Perché la gamma $[j, j + 2^i - 1]$ di lunghezza 2^i si divide bene negli intervalli $[j, j + 2^{i-1} - 1]$ e $[j + 2^{i-1}, j + 2^i - 1]$, entrambi di lunghezza 2^{i-1} , si può generare la tabella in modo efficiente utilizzando la programmazione dinamica:

```
1 std::copy(array.begin(), array.end(), st[0]);
2
3 for (int i = 1; i <= K; i++)
4     for (int j = 0; j + (1 << i) <= N; j++)
5         st[i][j] = f(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

La funzione f dipenderà dal tipo di query. Per le query con somma dell'intervallo calcolerà la somma, per le query con somma minima dell'intervallo calcolerà il minimo. La complessità temporale del precalcolo è $O(N \log N)$.

1.2.3 Query sulla soma degli intervalli

Per questo tipo di query, si vuole trovare la somma di tutti i valori in un intervallo. Quindi la definizione naturale della funzione f è $f(x, y) = x + y$. Si può costruire la struttura dati con:

```
1  long long st[K + 1][MAXN];
2
3  std::copy(array.begin(), array.end(), st[0]);
4
5  for (int i = 1; i <= K; i++)
6      for (int j = 0; j + (1 << i) <= N; j++)
7          st[i][j] = st[i - 1][j] + st[i - 1][j + (1 << (i - 1))];
```

Per rispondere alla query di somma per l'intervallo $[L, R]$, si itera su tutte le potenze di due, partendo da quella più grande. Non appena una potenza di due 2^i è minore o uguale alla lunghezza dell'intervallo ($= R - L + 1$), si elabora la prima parte dell'intervallo $[L, L + 2^i - 1]$, e si continua con l'intervallo rimanente $[L + 2^i, R]$.

```
1  long long sum = 0;
2  for (int i = K; i >= 0; i--) {
3      if ((1 << i) <= R - L + 1) {
4          sum += st[i][L];
5          L += 1 << i;
6      }
7  }
```

La complessità temporale per una query Range Sum è $O(K) = O(\log MAXN)$.

1.2.4 Query minime di intervallo (RMQ)

Sono queste le query in cui la tabella sparsa eccelle. Quando si vuole calcolare il minimo di un intervallo, non imposta se si elabora un valore nell'intervallo una o due volte. Pertanto, invece di suddividere un intervallo in più intervalli, lo si può anche suddividere in soli due intervalli sovrapposti con potenza di due lunghezze. Ad esempio, si può dividere l'intervallo $[1, 6]$, nelle gamme $[1, 4]$ e $[3, 6]$. L'intervallo minimo di $[1, 6]$ è chiaramente uguale al minimo dell'intervallo minimo di $[1, 4]$ e l'intervallo minimo di $[3, 6]$. Quindi si può calcolare il minimo dell'intervallo $[L, R]$ con.

$$\min(st[i][L], st[i][R - 2^i + 1])$$

dove $i = \log_2(R - L + 1)$. Ciò richiede di essere in grado di calcolare $\log_2(R - L + 1)$ velocemente. È possibile ottenere questo risultato precalcolando tutti i logaritmi:

```
1  int lg[MAXN+1];
2  lg[1] = 0;
3  for (int i = 2; i <= MAXN; i++)
4      lg[i] = lg[i/2] + 1;
```

Successivamente si devono precalcolare la struttura dati della tabella sparsa. Questa volta viene definito f con $f(x, y) = \min(x, y)$.

```
1  int st[K + 1][MAXN];
2
3  std::copy(array.begin(), array.end(), st[0]);
4
5  for (int i = 1; i <= K; i++)
6      for (int j = 0; j + (1 << i) <= N; j++)
7          st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
```

Il minimo di un intervallo $[L, R]$ può essere calcolato con:

```

1  int i = lg[R - L + 1];
2  int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);

```

La complessità temporale per una query minima di intervallo è $O(1)$.

1.2.5 Limitazioni

L'approccio sopra discusso supporta solo query di funzioni idempotenti. In altre parole, funziona benissimo per le query con l'intervallo minimo, ma non è possibile rispondere alle query con somma degli intervalli utilizzando questo approccio.

Esistono strutture dati simili in grado di gestire qualsiasi tipo di funzione associativa e rispondere a query di intervallo in $O(1)$. Una si chiama *tabella sparsa disgiunta*. Un'altra sarebbe l'*albero quadrato*.

1.3 Unione di insiemi disgiunti

Vengono dati diversi elementi, ognuno dei quali è un insieme separato. Una DSU ha un'operazione per combinare due insiemi qualsiasi e sarà in grado di dire in quale insieme si trova un elemento specifico. La versione classica introduce anche una terza operazione: può creare un insieme a partire da un nuovo elemento.

Pertanto l'interfaccia di base di questa struttura dati è composta da sole tre operazioni:

- **make_set(v)**: crea un nuovo set costituito dal nuovo elemento v ;
- **union_sets(a,b)**: unisce i due insiemi specificati (si trova l'insieme in cui è presente l'elemento a e si trova l'insieme in cui è presente l'elemento b);
- **find_set(v)**: restituisce il rappresentante (detto anche leader) dell'insieme che contiene l'elemento v . Questo rappresentante è un elemento del suo insieme corrispondente. Viene selezionato in ogni set della struttura dati stessa (e può cambiare nel tempo, vale a dire dopo **union_sets** chiamate). Questo rappresentante può essere utilizzato per verificare se due elementi fanno parte o meno dello stesso insieme. Due elementi a e b sono nello stesso insieme, se **find_set(a) == find_set(b)**. Altrimenti si trovano in set diversi.

La struttura dati consente di eseguire ciascuna di queste operazioni in quasi $O(1)$ di tempo in media.

1.3.1 Costruire una struttura dati efficiente

Si memorizzano i set sotto forma di alberi: ogni albero corrisponde a un insieme. E la radice dell'albero sarà il rappresentante/leader del set (fig. 4) All'inizio ogni elemento inizia come un

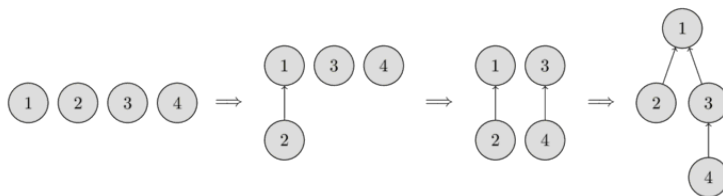


Figura 4: rappresentazione dei set.

unico insieme, quindi ogni vertice è un albero a sé stante. Si combinano l'insieme contenente l'elemento 1 e l'insieme contenente l'elemento 2. Poi si combinano l'insieme contenente l'elemento 3 e l'insieme contenente l'elemento 4 e nell'ultimo passaggio si combinano l'insieme contenente l'elemento 1 e l'insieme contenente l'elemento 3.

Per l'implementazione ciò significa che bisogna mantenere un array `parent` che memorizza nell'albero un riferimento al suo antenato immediato.

1.3.2 Implementazione ingenua

Questa è una prima implementazione, piuttosto inefficiente. Successivamente si applicheranno due ottimizzazioni in modo che ogni chiamata di funzione richieda un tempo pressoché costante. Tutte le informazioni sugli insiemi di elementi saranno conservate in un array `parent`.

Per creare un nuovo set (operazione `make_set(v)`), si crea semplicemente un albero con radice nel vertice `v`, il che significa che è un suo antenato.

Per combinare due set (operazione `union_sets(a,b)`), si trovano innanzitutto il rappresentante dell'insieme in cui `a` si trova, e il rappresentante dell'insieme in cui `b` si trova. Se i rappresentanti sono identici, e non si ha nulla da fare, gli insiemi sono già uniti. Altrimenti si può semplicemente specificare che uno dei rappresentanti è il genitore dell'altro rappresentante, combinando così i due alberi.

Infine nell'implementazione della funzione di ricerca rappresentativa (operazione `find_set(v)`) si scalano gli antenati del vertice `v` fino a raggiungere la radice, cioè un vertice tale che il riferimento all'antenato conduca a se stesso. Questa operazione è facilmente implementabile ricorsivamente.

```

1  void make_set(int v) {
2      parent[v] = v;
3  }
4
5  int find_set(int v) {
6      if (v == parent[v])
7          return v;
8      return find_set(parent[v]);
9  }
10
11 void union_sets(int a, int b) {
12     a = find_set(a);
13     b = find_set(b);
14     if (a != b)
15         parent[b] = a;
16 }

```

Tuttavia questa implementazione è inefficiente. È facile costruire un esempio in modo che gli alberi degenerino in lunghe catene. In tal caso ogni chiamata `find_set(v)` può prendere $O(n)$ tempo.

1.3.3 Ottimizzazione della compressione del percorso

È progettata per accelerare `find_set`. Se si chiama `find_set(v)` per qualche vertice `v`, si trova effettivamente il rappresentante `p` per tutti i vertici che si visitano sul percorso tra `v` e il rappresentante effettivo `p`. Il trucco è quello di rendere i percorsi per tutti quei nodi più brevi, impostando il genitore di ogni vertice visitato direttamente su `p`.

È possibile vedere l'operazione in fig. 5. A sinistra c'è un albero e sul lato destro c'è l'albero compresso dopo aver chiamato `find_set(7)`, che accorcia i percorsi per i nodi visitati 7,5,3 e 2. L'implementazione nuova di `find_set` è in elenco 17.

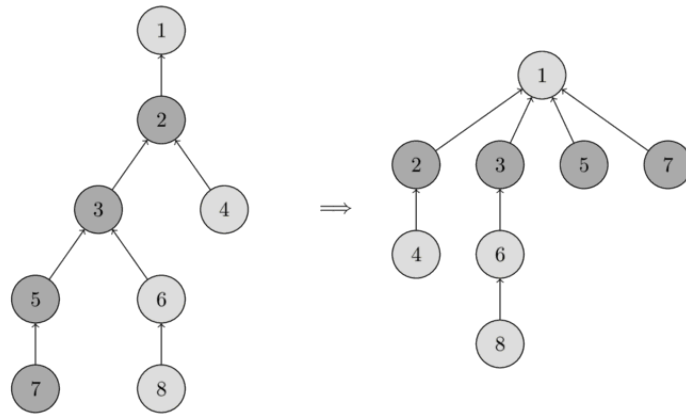


Figura 5: compressione del percorso.

Listing 17: nuova implementazione di `find_set`.

```

1  int find_set(int v) {
2      if (v == parent[v])
3          return v;
4      return parent[v] = find_set(parent[v]);
5  }

```